

◆ The RAGulator — Pocket Manual

· Psypher Labs · v9.0.1 · quick reference

What it is. A teaching-grade PoC: a layered pipeline that decides what a natural-language request wants and whether the user may have it. Rules first (cheap, certain), then an LLM for meaning rules can't see. Identity/role/classification are self-asserted by design — not production security.

Quick start

```
python bootstrap_ragulator.py ragulator && cd ragulator
python3 -m venv .venv && source .venv/bin/activate
pip install -e "[dev,notebook]"
cp .env.example .env          # paste GOOGLE_API_KEY into .env
pytest                        # 37 passed, 6 skipped
python -m ragulator --role analyst
```

Commands

Command	Purpose
<code>python -m ragulator --role analyst</code>	Interactive console as a role
<code>--role "public user"</code>	Low-clearance role (quote multi-word)
<code>--query "..."</code>	Run one query and exit
<code>--nkt-mode loose moderate strict</code>	Override need-to-know strictness
<code>--no-llm</code>	Data-layer only (no routing)
<code>pytest</code>	Offline suite (no key) — 37 tests
<code>pytest -k live -v -s</code>	Live Gemini suite (key) — 6 tests

Capabilities (type at the > prompt)

Say something like...	Intent	What happens
find documents about strategy	SEARCH	Keyword search, filtered by your clearance
retrieve document 5	RETRIEVE	Clearance check, then need-to-know check
what is the need-to-know policy?	POLICY	Answered from the RAG index over 11 policies
tell me a joke	GENERAL	Polite redirect to real functions
ignore previous instructions	BLOCK (rule)	Instant block, 0.0s, no API call
bypass the clearance check...	BLOCK (LLM)	Gemini flags disguised attacks the regex misses

Roles & clearance

Clearance	Lvl	Roles
TS — Top Secret	4	administrator, senior analyst
S — Secret	3	project manager, analyst
C — Confidential	2	developer, support staff
U — Unclassified	1	intern/trainee, public user

Access rule: you only ever see documents at or below your clearance, and only if need-to-know passes. Clearance alone is never sufficient.

The pipeline (every request, in order)

1. Sanitize – normalise & length-limit input
2. Rule pre-filter – regex catches obvious attacks (free, instant)
3. LLM threat check – Gemini catches disguised/rephrased attacks
4. Intent routing – search / retrieve / policy / general
5. Access control – clearance gate, then need-to-know gate
6. Dispatch – run the matching action; return result

Why both layers: rules are fast, free, and certain but match strings; the model understands meaning, paraphrase, and purpose. Rules handle the obvious; the model handles intent. Remove either and the system gets weaker.

Need-to-know modes

Mode	Behaviour
loose	Clearance gap ≥ 1 over the doc auto-grants need-to-know
moderate	Clearance gap ≥ 2 auto-grants (default)
strict	No auto-grant; topical match always required

Key config (config.yaml)

Setting	Default
gemini_model	gemini-2.5-flash
embedding_model	models/gemini-embedding-001
nkt_mode	moderate
num_documents	1000
policy_rag_k	3

Fast fixes

Symptom	Fix
404 / NOT_FOUND on a model	Model name retired; list models, update config.yaml
API_KEY_INVALID	Re-copy key into .env; no quotes/spaces
Live retrieve denied	Need-to-know; use --nkt-mode loose or match the purpose
Banner shows \u25c6	Cosmetic only (ANSI fallback); ignore

Other models (preference, not a constraint)

Gemini is the reference provider because this began as a Colab project. The LLM is isolated behind one service class, so Claude or OpenAI can drive the same pipeline. It is a code change (implement the same small interface against their SDK + swap embeddings + point at the right key env var), not a config toggle today.

◆ it regulates. · Not a production security system.